

CineCart: A Cinema Booking Cart

Object-Oriented Programming — Lab 4

Marks: 15 **Language:** Java (JDK 17) **Approach:** Test-Driven

Submit your .java source files only. Do *not* modify the test file or CsvLoader.java.

What this lab teaches

This lab combines everything you have built so far – classes, constructors, types, operators, loops, conditionals, and arrays of objects – into a small twelve-class system spread across multiple Java packages. You will read input from CSV files using a *provided* loader, compose objects out of other objects, and let one class delegate work to another. Methods you write here will contain non-trivial branching and arithmetic, so passing the test suite requires more than matching method signatures.

Getting Started: Git and IDE Setup

The course Git repository has one branch per student plus a master branch that contains the project skeleton. Before you write any code, do the following from inside the cloned repository folder.

```
# 1. Identify yourself for this repository (use YOUR name and student email).
git config user.name "Your Full Name"
git config user.email "yourid@university.edu"

# 2. Check which branch you are on right now.
git branch                # the * marks your current branch

# 3. If you are NOT on your own student branch, switch to it.
git checkout <your-student-branch>

# 4. Pull the latest commits on YOUR branch from the server.
git pull origin <your-student-branch>

# 5. Bring in the project skeleton from master into your branch.
git merge origin/master      # resolve any conflicts if prompted
```

Open in IntelliJ. After the merge succeeds, your repository contains a folder named CineCart/ which holds build.gradle and settings.gradle. *This* is the folder you open in IntelliJ IDEA: choose *File* → *Open...*, navigate to the CineCart/ folder (not the repository root, not the src/ folder), and let IntelliJ import it as a Gradle project. IntelliJ will detect the Gradle wrapper, download dependencies, and set up the run/test configurations automatically.

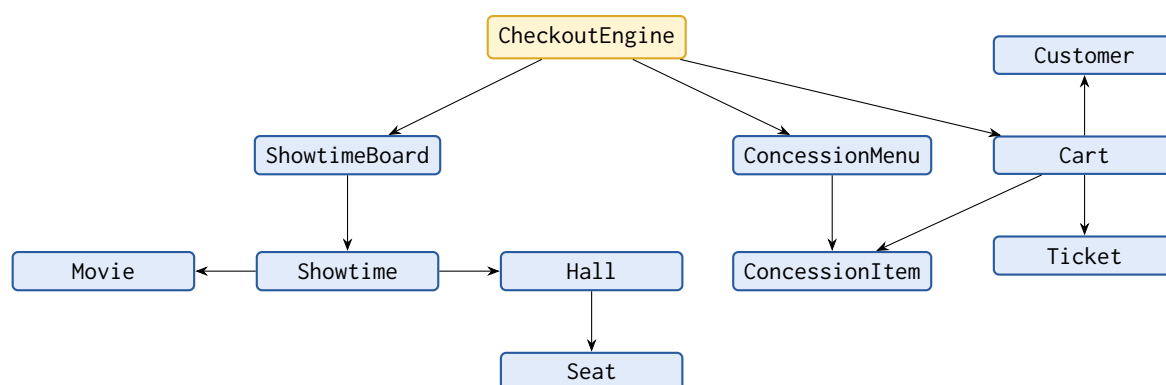
Run the tests. From IntelliJ's Gradle tool window, run the test task. From the terminal you can run ./gradlew test (Linux/macOS) or gradlew.bat test (Windows) inside the CineCart/ folder. All tests will fail initially – that is expected; pass them one by one as you implement each part.

The Scenario

Cineplex Bashundhara is a small multiplex with a few halls and a snack counter. When a customer walks up to the booking booth, they pick a showtime, get age-checked against the movie's rating, choose a seat from a 2D grid in the assigned hall, and may also add concession items (popcorn, soda, nachos) to the same cart. At the end, the booth runs a single checkout pipeline that applies seat, peak-hour, loyalty-tier, group, and combo rules and prints a receipt.

You will build a solution that power this booth. The CSV reader is given to you; you must *not* modify it. The data files live in a folder called `test-data/`.

Class diagram. Below is the relationship map. `Cart` *owns* an array of `Tickets` and an array of `ConcessionItems`. `Hall` *owns* a 2D grid of `Seats`. `Showtime` *points to* a `Movie` and a `Hall`. `CheckoutEngine` drives the whole flow.



Folder layout (Gradle). The project follows the standard Gradle layout. You must place each class in the correct package; the package declaration at the top of every `.java` file must match the folder it lives in.

```

CineCart/                                     <- open THIS folder in IntelliJ
|-- build.gradle                             settings.gradle
|-- gradlew                                  gradlew.bat
|-- gradle/wrapper/                          gradle-wrapper.jar  gradle-wrapper.properties
|-- .gitignore
+-- src/
    +-- main/
        | +-- java/
        | | +-- model/    Movie.java Seat.java Hall.java Showtime.java
        | | |             Customer.java Ticket.java ConcessionItem.java
        | | |             Cart.java
        | | +-- data/     ShowtimeBoard.java ConcessionMenu.java
        | | +-- engine/   CheckoutEngine.java
        | | +-- io/       CsvLoader.java          <- provided, do not edit
        | | +-- app/      CineCartApp.java
        | +-- resources/
        |     movies.csv halls.csv showtimes.csv concessions.csv customers.csv
        csv
    +-- test/
        +-- java/          CineCartTest.java          <- provided, do not edit
  
```

The CSV files live under `src/main/resources/`. This is the standard Gradle/Maven location for static data assets that ship with the application: at build time they end up on the classpath, and the provided `CsvLoader` reads them via `getClass().getResourceAsStream(...)` so the same code works from IntelliJ, from `./gradlew run`, and from a packaged JAR.

▷ Notice

Constraint for all parts: use plain fixed-size arrays only. You may *not* import `java.util.ArrayList`, `HashMap`, `List`, or any other collection class. Every variable-length container must be a Java array (e.g. `Ticket[]`) tracked by a separate `int` count, just as in Labs 1, 2, and 3.

Part A — Movie, Seat, Hall

[3 marks]

These three classes describe what is shown and where. Hall is the first place in the course where you build a 2D array of objects.

A.1 Movie

Fields: `int id`, `String title`, `String rating`, `int durationMin`, `double basePrice`.

Constructor: All instance variables should be set.

Methods: getters for every field, plus

- `int getMinAge()` — returns 0 for "G", 7 for "PG", 13 for "PG-13", and 18 for "R".
- `String toString()` — returns e.g. "Inception (PG-13) 148min - BDT 350.00".

A.2 Seat

Fields: `int row`, `int col`, `boolean isPremium`, `boolean isBooked`.

Constructor: All instance variables should be set. Except a Seat should not be booked when its object is instantiated.

Methods: getters, `void book()`, `void release()`, `boolean isAvailable()`, and `toString()` that returns "R3C5" with a trailing * if premium, # if booked.

A.3 Hall

Fields: `int id`, `int rows`, `int cols`, `Seat[][] grid`.

Constructor: `Hall(int id, int rows, int cols, int premiumRows)` — builds the grid; for each row index `r` and column index `c`, create a Seat with `isPremium = (r < premiumRows)`.

Methods:

- `Seat getSeat(int row, int col)` — direct lookup.
- `int countAvailable()` — nested loop over the grid, counting seats whose `isAvailable()` is true.
- `void displayLayout()` — prints an ASCII map of the hall: . for available, # for booked, * for premium-available.

Part B — Showtime, Customer, Ticket, ConcessionItem [3 marks]

B.1 Showtime

Fields: `int id`, `Movie movie`, `Hall hall`, `int startHour` (0-23), `String dateTag`.

Constructor: full five-argument constructor.

Methods: getters; `boolean isPeak()` returning true when `startHour` is between 18 and 21 inclusive; `toString()` e.g. "[T17] Inception @ Hall 2 - 19:00 (Fri)".

B.2 Customer

Fields: `int id`, `String name`, `int age`, `String loyaltyTier` (one of "BASIC", "SILVER", "GOLD").

Constructors:

- `Customer(int id, String name, int age)` — defaults `loyaltyTier` to "BASIC".
- `Customer(int id, String name, int age, String tier)` — explicit tier.

Methods: getters; `double getTierDiscount()` that returns 0.15 for GOLD, 0.08 for SILVER, and 0.00 otherwise; `toString()`.

B.3 Ticket

Fields: Showtime showtime, int row, int col, double pricePaid.

Constructor: four-argument full constructor.

Methods: getters; toString() e.g. "T17 - R3C5 @ BDT 455.00".

B.4 ConcessionItem

Fields: String code, String name, double unitPrice.

Constructor: three-argument full constructor.

Methods: getters; toString() e.g. "POP - Popcorn (Large) - BDT 220.00".

Part C — Cart

[2 marks]

Cart is owned by one Customer and accumulates both Tickets and ConcessionItems as the visit progresses.

Constants: public static final int MAX_TICKETS = 20; public static final int MAX_ITEMS = 20.

Fields:

- Customer owner
- Ticket[] tickets
- int ticketCount
- ConcessionItem[] items
- int[] qtys
- int itemCount

Constructor: Cart(Customer owner) — allocate both arrays, set both counts to 0.

Methods:

- void addTicket(Ticket t) — append to tickets unless full.
- void addItem(ConcessionItem c, int qty) — append to items and qtys; reject if full or qty <= 0.
- Getters: getOwner(), getTickets(), getTicketCount(), getItems(), getQtys(), getItemCount().
- double sumTicketsPaid() — loop sum of every ticket's pricePaid.
- double sumConcessionsRaw() — loop sum of unitPrice * qty over all items.
- boolean hasItem(String code) — linear search by item code.

Part D — Data Layer

[1 mark]

These two classes are array-of-object containers that the loader fills and the engine queries. Both follow the exact same pattern.

D.1 ShowtimeBoard

Fields: Showtime[] shows, int count, static final int MAX = 30 (private).

Methods: void add>Showtime s); Showtime findById(int id) — linear search, returns null when not found; void displayAll() that prints every showtime via its toString().

D.2 ConcessionMenu

Fields: ConcessionItem[] items, int count, static final int MAX = 30 (private).

Methods: void add(ConcessionItem c); ConcessionItem findByCode(String code) — returns null when not found; void displayAll().

Why a separate class? A bare Showtime[] works, but every caller would have to remember its own count and re-write the lookup loop. ShowtimeBoard packages the array, the count, and the lookup together — the same lesson you learned moving from parallel arrays to a class in Lab 1.

Part E — CheckoutEngine: Booking Flow

[3 marks]

CheckoutEngine is the brain of the booth. It does not store any tickets or items itself — those belong to the customer's Cart. Instead it holds references to the read-only data layer (ShowtimeBoard, ConcessionMenu) and orchestrates the booking and pricing rules.

Fields (all private): ShowtimeBoard board, ConcessionMenu menu.

Constructor: CheckoutEngine>ShowtimeBoard board, ConcessionMenu menu).

E.1 bookTicket

Signature: String bookTicket(Cart cart, int showtimeId, int row, int col).

The method must perform these checks *in order* and must produce the side effects described:

1. Look up the showtime via board.findById(showtimeId). If it does not exist, return the string "Showtime not found" and change nothing.
2. Compare cart.getOwner().getAge() against showtime.getMovie().getMinAge(). If the customer is too young, return the string "Underage for rating <X>" (where <X> is the movie's rating) and change nothing – in particular, the seat must *not* be booked.
3. Look up the seat via showtime.getHall().getSeat(row, col). If the seat is already booked, return "Seat unavailable" and change nothing.
4. Compute the ticket price as

$$\text{price} = \text{movie.basePrice} \times (\text{seat.isPremium} ? 1.30 : 1.00) \times (\text{showtime.isPeak}() ? 1.20 : 1.00).$$

5. Mark the seat booked, build a Ticket with the computed price, and add it to the cart.
6. Return the string "OK".

E.2 addConcession

Signature: String addConcession(Cart cart, String code, int qty).

1. Look up the item via menu.findByCode(code). If null, return "Item not found".
2. If qty <= 0, return "Invalid quantity".
3. Otherwise add the item to the cart and return "OK".

Part F — CheckoutEngine: Checkout Pipeline

[2 marks]

F.1 checkout

Signature: double checkout(Cart cart).

Apply every rule below *in this order*; the method must return the final amount rounded to two decimal places.

1. ticketSubtotal = cart.sumTicketsPaid().
2. concessionSubtotal = cart.sumConcessionsRaw().
3. Combo deal: if the cart contains both POP and SODA, set combo = 50.0, otherwise combo = 0.0.
4. preDiscount = ticketSubtotal + concessionSubtotal – combo.
5. Group discount: if the cart contains *four or more* tickets, group = 0.10 × preDiscount, otherwise 0.

6. Tier discount: $\text{tier} = \text{cart.getOwner().getTierDiscount()} \times \text{preDiscount}$.
7. $\text{afterDiscounts} = \text{preDiscount} - \text{group} - \text{tier}$.
8. $\text{tax} = 0.05 \times \text{afterDiscounts}$.
9. Return $\text{round2}(\text{afterDiscounts} + \text{tax})$.

F.2 `getReceipt`

Signature: `String getReceipt(Cart cart)`.

Return a multi-line string. The exact wording is up to you, but the test will check that the string contains *all* of the following substrings: "Receipt", the customer's name, "BDT", "Total", and "Discount". Listing each ticket and each concession line is recommended.

▷ Notice

You may notice that `checkout` and `bookTicket` are getting long. That is fine for this lab — focus on getting the rules right and the tests passing. Resist the urge to introduce inheritance, interfaces, or design patterns; you have not been taught those yet, and this lab does not need them.

The Provided CSV Loader and Test Data

The course staff have written `io/CsvLoader.java`. You may not modify this file. Each loader method takes a *resource name* – the bare filename of a CSV that lives under `src/main/resources/` – and returns a fully populated array or container.

CineCartApp is your starting point. The class `app/CineCartApp.java` is the *main class* of the project: it contains the public static void `main(String[] args)` method that the JVM invokes when you run the application (via `./gradlew run`, the green play arrow in IntelliJ, or by executing the packaged JAR). Nothing in the rest of your code runs unless `main` calls into it. Your task in `CineCartApp` is to wire the system together end-to-end and demonstrate that it works: you will load the CSV data, create a customer with a cart, perform a few bookings, add some concessions, run the checkout, and print the receipt to standard output.

Why call the CSV loader? The `Movie`, `Hall`, `Showtime`, `ConcessionItem`, and `Customer` objects do not appear out of thin air – their data lives in the five CSV files inside `src/main/resources/`. The loader's job is to read each line of every CSV, parse the fields into the correct Java types, build the corresponding objects, and hand back ready-to-use arrays (or, for showtimes and concessions, ready-to-use container classes). Without this step you would have no movies for the booth to sell, no halls to seat customers in, no showtimes for them to choose from, and no menu of snacks. Calling the loader is therefore always the *first* thing `main` does; everything else — creating a `CheckoutEngine`, opening a `Cart`, calling `bookTicket` — depends on the data being in memory first.

Sketch of what CineCartApp should do. Treat the body of `main` as a short script that reads top-to-bottom and tells a story:

1. **Load the data.** Call each of the loader methods on the five resource files in turn and keep the returned arrays/containers in local variables.
2. **Construct the engine.** Instantiate one `CheckoutEngine`, passing it the populated `ShowtimeBoard` and `ConcessionMenu`.
3. **Pick a customer and open a cart.** Choose one of the loaded customers (or create a new one) and instantiate a `Cart` for them.
4. **Display the catalogue.** Print the available showtimes (using `ShowtimeBoard.displayAll`) and, optionally, the seat layout of the chosen hall so the user can see what is on offer.
5. **Book a few tickets.** Call `engine.bookTicket(...)` two or three times with different showtime ids, rows, and columns. Print the return value each time so the customer knows whether the booking succeeded or was rejected (underage, seat taken, unknown showtime, etc.).
6. **Add concessions.** Call `engine.addConcession(...)` once or twice – a popcorn-and-soda combo is a good demonstration because it triggers a discount in checkout.
7. **Check out.** Call `engine.checkout(cart)` to obtain the final price and `engine.getReceipt(cart)` to obtain the printable summary, then write the receipt to `System.out`.

The loader call itself looks like this – this is the only piece of `CineCartApp` you are given verbatim; the rest of the script you compose yourself based on the steps above:

```
import io.CsvLoader;
import model.*;
import data.*;

Movie[]  movies  = CsvLoader.loadMovies  ("movies.csv");
Hall[]   halls   = CsvLoader.loadHalls   ("halls.csv");
```

```
ShowtimeBoard board = CsvLoader.loadShowtimes(
    "showtimes.csv", movies, halls);
ConcessionMenu menu = CsvLoader.loadConcessions("concessions.csv");
Customer[] customers = CsvLoader.loadCustomers("customers.csv");
```

CSV file shapes. Every file has a header row, comma-separated, no embedded commas in fields.

```
movies.csv      id,title,rating,durationMin,basePrice
halls.csv       id,rows,cols,premiumRows
showtimes.csv   id,movieId,hallId,startHour,dateTag
concessions.csv code,name,unitPrice
customers.csv   id,name,age,tier
```

Sample rows.

```
movies.csv:
1,Inception,PG-13,148,350.00
2,The Godfather,R,175,300.00
3,Toy Story 4,G,100,250.00

halls.csv:
1,5,8,1
2,6,10,2

showtimes.csv:
17,1,2,19,Fri
18,2,1,21,Fri
19,3,1,11,Sat

concessions.csv:
POP,Popcorn (Large),220.00
SODA,Soda (Large),120.00
NACHO,Nachos,180.00

customers.csv:
1,Alice,28,GOLD
2,Bob,16,BASIC
3,Carol,35,SILVER
```

Marking Rubric

Part	Criterion	Marks
A	Movie, Seat, Hall tests pass	3
B	Showtime, Customer, Ticket, ConcessionItem tests pass	3
C	Cart tests pass	2
D	ShowtimeBoard, ConcessionMenu tests pass	1
E	bookTicket + addConcession tests pass	3
F	checkout + getReceipt tests pass	2
All	Correct package layout; CsvLoader unmodified	1
Total		15

General Notes

- Use double for prices, multipliers, and tax. Use int for ages, hours, rows, columns, ids.
- Round monetary totals to two decimals only at the very end of checkout: `Math.round(value * 100.0) / 100.0`.
- Format numbers in receipts with `String.format("%.2f", value)`.
- Do *not* import `java.util.ArrayList`, `HashMap`, or any other collection class.
- Build and test from the terminal with `./gradlew test` (or `gradlew.bat test` on Windows). From IntelliJ, use the green play arrow next to the test task in the Gradle tool window, or right-click `CineCartTest` in the project tree.
- Partial credit is awarded. Submit whatever compiles, even if some methods are incomplete.
- **Submission:** commit and push your work to your student branch. Do *not* commit `build/`, `.gradle/`, or anything from `.idea/`; the existing `.gitignore` already excludes them. Do not modify `CsvLoader.java`, `CineCartTest.java`, the CSV files in `src/main/resources/`, or `build.gradle`.